

Candidate Profile Shortlisting System using API & Skill Matching (with OpenRouter AI)

Problem Statement

Design and implement a Candidate Shortlisting System that filters and ranks candidates based on required skill sets. The system should also integrate an AI API (OpenRouter) to intelligently suggest best-fit candidates and improve matching accuracy.

Functional Requirements

1. Candidate Management

Store candidate details:

Name

Email

Skills (array)

Experience (years)

Projects / Bio

2. Job Requirement Input

Recruiter can input:

Required skills

Minimum experience

Preferred skills

3. Shortlisting Logic (Core Feature)

Match candidates based on:

Skill overlap (%)

Experience criteria

Rank candidates:

High match → Top

Partial match → Medium

Low match → Bottom

4. AI-Based Shortlisting (OpenRouter)

Use OpenRouter API to:

Analyze candidate profiles

Suggest top candidates

Explain why a candidate is suitable

Improve ranking beyond simple keyword matching

 Required API Endpoints

 Candidate APIs

1. Add Candidate

POST /api/candidates

Body:

```
{  
  "name": "Rahul Sharma",  
  "email": "rahul@gmail.com",  
  "skills": ["React", "Node.js", "MongoDB"],  
  "experience": 2  
}
```

2. Get All Candidates

GET /api/candidates

 Job Matching APIs

3. Shortlist Candidates (Basic Logic)

POST /api/match

Body:

```
{  
  "requiredSkills": ["React", "Node.js"],  
  "minExperience": 1  
}
```

4. AI-Based Candidate Suggestion

POST /api/ai/shortlist

 OpenRouter API Integration

◆ Example Prompt for AI Ranking

```
const response = await fetch("https://openrouter.ai/api/v1/chat/completions", {  
  method: "POST",  
  headers: {  
    "Authorization": "Bearer YOUR_API_KEY",  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({  
    model: "openai/gpt-5.2",  
    messages: [  
      {  
        role: "user",  
        content: `  
Job requires: React, Node.js (2+ years experience)  
  
Candidates:  
  
1. Rahul - React, Node.js, MongoDB - 2 years  
2. Ankit - HTML, CSS - 1 year  
3. Priya - React, Node.js, AWS - 3 years
```

Rank candidates and explain why.

```
,  
}  
]  
})  
});
```

🔗 Backend Matching Logic (Basic)

```
function matchCandidates(candidates, job) {  
  return candidates.map(candidate => {  
    const matchedSkills = candidate.skills.filter(skill =>  
      job.requiredSkills.includes(skill)  
    );  
  
    const score = matchedSkills.length / job.requiredSkills.length;  
  
    return {  
      ...candidate,  
      matchScore: score  
    };  
  }).sort((a, b) => b.matchScore - a.matchScore);  
}
```

😊 Frontend (React)

Pages / Components:

Candidate Form (Add Candidate)

Job Requirement Form

Candidate List

Shortlisted Candidates Display

Output Display

Display:

Candidate Name

Match Score (%)

Skills Matched

AI Recommendation (text explanation)

Database Schema (MongoDB)

```
const CandidateSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  skills: [String],  
  experience: Number,  
  createdAt: { type: Date, default: Date.now }  
});
```

Bonus Features

 Search & filter candidates

 Graph showing match scores

 AI-generated interview questions

 Save shortlisted candidates